Sitzung 2: rse_annotations – Forschungscode markieren und prüfen

Grundlagen: Immutabilität und Dataclasses

Dr. Julian Dehne

Gesellschaft für Informatik e.V. (GI) · deRSE
julian.dehne@gi.de · ORCID 0000-0001-9265-9619

Immutabilität

Was heißt unveränderlich?

- Ein **unveränderliches** (immutable) Objekt wird nach seiner Erzeugung nicht mehr geändert.
- Eine „Änderung” erzeugt ein **neues** Objekt; das alte bleibt, wie es war.

	unveränderlich	veränderlich
Python	`int`, `str`, `tuple`, `frozenset`	`list`, `dict`, `set`
Java	`String`, `record`, `List.of(...)`	`ArrayList`, `HashMap`

```
s = "abc"
t = s.upper()       # neues Objekt; s ist weiterhin "abc"
```

Warum im Werkzeug?

- `rse_annotations` hält jeden Concern in einer globalen Tabelle `CONCERNS`. Dasselbe Objekt ist zugleich **Decorator** und **Detektor** und wird überall geteilt.
- Jeder Detektor bekommt dieselbe `FunctionView` einer Funktion. Ein Detektor darf nicht verändern, was der nächste sieht.
- Unveränderliche Objekte sind **hashbar**: Sie können Schlüssel in `dict` oder Elemente in `set` sein.

Was nach der Erzeugung nicht mehr geändert werden kann, muss man beim Lesen nicht verfolgen.

frozen=True in Python

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Concern:
    name: str
    help: str

c = Concern("stochastic", "Ist der Zufall geseedet?")
c.name = "random"
# dataclasses.FrozenInstanceError: cannot assign to field 'name'
```

- Jede Zuweisung an ein Feld wirft `FrozenInstanceError`.
- `frozen=True` erzeugt zusätzlich `__hash__` aus den Feldern.

Grenze: nur flach (shallow)

```python
@dataclass(frozen=True)
class FunctionView:
    reads: list[str]

v = FunctionView(["open"])
v.reads = []             # FrozenInstanceError
v.reads.append("load")   # erlaubt: die Liste ist veränderlich
```

- `frozen` schützt die **Felder**, nicht die Objekte, auf die sie zeigen.
- Echte Unveränderlichkeit: `tuple` statt `list`, `frozenset` statt `set`.
- Im Prototyp hält `FunctionView` noch Listen – geschützt nur durch die Konvention „Detektoren lesen nur”.

Werte im Konstruktor anpassen

```python
@dataclass(frozen=True)
class FunctionView:
    reads: tuple[str, ...]

    def __post_init__(self):
        # self.reads = ... wäre ein FrozenInstanceError
        object.__setattr__(self, "reads", tuple(sorted(self.reads)))
```

- `__post_init__` läuft nach dem generierten `__init__`.
- Innerhalb der Klasse geht Zuweisen nur über `object.__setattr__`. Das ist eine bewusste Hintertür, nur für die Konstruktion.

Vergleich: Java

```java
public record FunctionView(String name, List<String> reads) {
    public FunctionView {                    // compact constructor
        reads = List.copyOf(reads);          // unveränderliche Kopie
    }
}
```

- Ein `record` hat nur `final`-Felder und generiert Konstruktor, Getter, `equals`, `hashCode`, `toString`.
- Auch hier flach: Ohne `List.copyOf` könnte der Aufrufer die Liste später ändern.
- Python `__post_init__` mit `object.__setattr__` entspricht dem *compact constructor*.

Dataclasses

Was `@dataclass` generiert

Von Hand

```
class Concern:
    def __init__(self, name, help):
        self.name = name
        self.help = help

    def __repr__(self):
        return f"Concern({self.name!r})"

    def __eq__(self, other):
        return (self.name, self.help) == \
               (other.name, other.help)
```

Mit Dataclass

```
from dataclasses import dataclass

@dataclass
class Concern:
    name: str
    help: str
```

- `__init__`, `__repr__`, `__eq__` aus den annotierten Feldern
- mit `frozen=True` zusätzlich `__hash__`

Felder steuern mit *field()*

```python
from dataclasses import dataclass, field

@dataclass
class Record:
    qualname: str
    marked: list[str] = field(default_factory=list)
    source: str = field(default="", repr=False)
```

- **Defaults**: Felder mit Default müssen nach denen ohne stehen.
- `default_factory=list`: jede Instanz bekommt eine eigene Liste (nie `= []`).
- `repr=False`: das Feld erscheint nicht in der Ausgabe, z.B. langer Quelltext oder eine Funktion.

Beispiel: *Concern* im Prototyp

```python
@dataclass(frozen=True)
class Concern(ABC):
    name: str
    help: str

    @abstractmethod
    def detect(self, v: FunctionView) -> Optional[Evidence]: ...
    def __call__(self, fn=None, **meta):      # die Instanz *ist* der Decorator
        ...

stochastic = Stochastic("stochastic", "Ist der Zufall geseedet?")
@stochastic
def sample(papers): ...
```

- Zwei Felder: Name und Frage an die Prüferin. `detect` schreibt jede Unterklasse.
- `frozen`: Kein Plugin kann einen Concern zur Laufzeit umbauen.

Beispiel: *FunctionView* und eine Fabrikmethode

```
@dataclass(frozen=True)
class FunctionView:
    name: str
    params: List[str]
    reads: List[str]
    writes: List[str]
    called: Set[str]
    ...

    @classmethod
    def of(cls, node: ast.AST) -> "FunctionView":
        ...                     # AST einmal durchlaufen
        return cls(name=..., params=..., reads=..., ...)
```

- `of` ist eine **Fabrikmethode**: Die Arbeit (AST lesen) steht dort, der generierte `__init__` speichert nur.
- Java: statische Methode `FunctionView.of(node)` vor dem Konstruktor des Records.

Kopie statt Änderung

```python
from dataclasses import replace

c2 = replace(c, help="Ist der Seed dokumentiert?")
# c bleibt unverändert, c2 ist ein neues Objekt
```

- `dataclasses.replace` erzeugt eine Kopie mit geänderten Feldern.
- Java: *Wither*-Methoden (`withHelp(...)`) oder Lombok `@Value`/`@Builder` mit `toBuilder()`.

Abstrakte Klassen

Eine Methode ohne Rumpf

```python
from abc import ABC, abstractmethod

class Concern(ABC):
    @abstractmethod
    def detect(self, v):
        """Liefert (Grund, Konfidenz) oder None."""
```

- `ABC` macht die Klasse **abstrakt**; `@abstractmethod` markiert die Methode, die jede Unterklasse schreiben **muss**.
- Der Rumpf ist leer, nur ein Docstring oder `...`: Er beschreibt den Vertrag, nicht die Umsetzung.
- Die Basisklasse legt fest, *was* ein Concern ist (Name, Frage, Decorator). *Wie* er erkannt wird, entscheidet die Unterklasse.

Unterklasse implementiert *detect*

```
class Stochastic(Concern):
    def detect(self, v):
        hits = sorted(v.called & RNG_CALLS)
        if not hits:
            return None
        if _seed(v):
            return f"nutzt {hits[0]}, Seed als Parameter", "high"
        return f"nutzt {hits[0]} ohne Seed-Parameter", "high"

CONCERN = Stochastic("stochastic", "Ist der Zufall geseedet?")
```

- Jeder Concern-Ordner hat eine `concern.py` mit genau einer solchen Unterklasse und der Instanz `CONCERN`.
- `registry.py` lädt alle `concern.py` und baut daraus die Tabelle `CONCERNS`.

Was passiert, wenn `detect` fehlt?

```
class Vergessen(Concern):
    pass

Vergessen("vergessen", "?")
# TypeError: Can't instantiate abstract class Vergessen
#            without an implementation for abstract method 'detect'
```

- Der Fehler kommt beim **Erzeugen**, nicht erst beim ersten Aufruf von `detect`.
- Mit `raise NotImplementedError` statt `@abstractmethod` würde das Plugin geladen und erst mitten im Scan scheitern.
- Python prüft das zur Laufzeit beim Laden der Registry, Java schon beim Kompilieren.

Vergleich: Java

Abstrakte Klasse

```
public abstract class Concern {
    final String name, help;

    Concern(String name, String help) {
        this.name = name;
        this.help = help;
    }

    // kein Rumpf
    abstract Optional<Evidence>
        detect(FunctionView v);
}
```

Interface

```
interface Detector {
    Optional<Evidence>
        detect(FunctionView v);
}
```

- Python kennt kein `interface`; eine `ABC` nur aus abstrakten Methoden spielt dieselbe Rolle.
- Python erlaubt Mehrfachvererbung, also auch mehrere ABCs.

Dataclass und ABC zusammen

```python
@dataclass(frozen=True)
class Concern(ABC):
    name: str
    help: str

    @abstractmethod
    def detect(self, v): ...

class Stochastic(Concern):          # kein eigenes @dataclass nötig
    def detect(self, v): ...

Stochastic("stochastic", "Ist der Zufall geseedet?")   # geerbtes __init__
```

- Die Unterklasse **erbt** das generierte `__init__`, `__repr__`, `__eq__` und `frozen`.
- Ein Java-`record` kann weder abstrakt sein noch erben. In Python lassen sich Wertklasse und abstrakte Basisklasse verbinden.

Python und Java im Überblick

	Python	Java
Wertklasse	`@dataclass(frozen=True)`	`record`
Beim Erzeugen anpassen	`__post_init__`	compact constructor
Unveränderliche Sammlung	`tuple`, `frozenset`	`List.copyOf`, `Set.copyOf`
Fabrikmethode	`@classmethod of(...)`	`static of(...)`
Kopie mit Änderung	`dataclasses.replace`	Wither, `toBuilder()`
Abstrakte Methode	`@abstractmethod`	`abstract`
Als Decorator nutzbar	ja (`__call__`)	nein (Annotation + Record)